

OneLife

Repo-based app summary, architecture identity, and instruction comparison

Built only from repo evidence in the current checkout. When information is missing or unverifiable from the repo, this document marks it as "Not found in repo."

Primary form	A SwiftUI life simulation app with a yearly turn loop, a planner-style tab UI, origin setup at age 14, and domain-driven state for education, career, finance, relationships, health, housing, assets, and progress.
Core loop	Preview origin -> begin life -> pick yearly focus actions -> age up -> resolve systems -> surface a weighted event -> apply choice effects -> save progress.
Evidence base	ContentView.swift, Persistence.swift, Models.swift, EventEngine.swift, SampleEvents.json, OneLifeApp.swift, OneLifeBaseApp.swift.

Evidence Snapshot

- UI shell: ContentView uses a NavigationStack, tab strip, "Life Signals," per-domain planner tabs, an event sheet, and a persistent Age Up dock.
- Simulation core: LifeSimulationOrchestrator sequences yearly updates across Trait, Policy, Education, Career, Finance, Housing, Relationships, Health, Assets, and Progress systems.
- Data model: GameState aggregates player and domain states; events are loaded from SampleEvents.json.
- Persistence: PersistenceCoordinator saves a versioned envelope, maintains a backup file, supports legacy save recovery, and records DEBUG timing metrics.
- Event content observed: 24 sample events across general, education, career, finance, relationships, and health categories.

What It Is

OneLife is a planner-shaped life sim. The current repo shows a game that starts by generating or selecting a compressed origin, lands the player at age 14, then moves through life one year at a time. Each year is framed as a compact planning cycle: review top signals, choose a focus in each active life area, press Age Up, then absorb the consequences.

Who It Is For

Explicit target audience	Not found in repo.
Repo-evidence interpretation	Players who want a compact life simulation centered on balancing school or work, money pressure, social ties, and health over time rather than controlling minute-to-minute actions.

What It Does

- Lets the player start from either a weighted random origin or one of seven origin templates.
- Tracks domain state for education, career, finance, relationships, health, housing, assets, and long-term progress.
- Offers yearly focus actions for five action domains: education, career, finance, relationships, and health.
- Runs a sequential yearly simulation pass, then surfaces a weighted event chosen from JSON-authored content.
- Applies event choice effects across core stats and domain-specific state, records the outcome to history, and unlocks milestones.
- Persists progress to disk with primary and backup saves, plus backward-compatible legacy loading paths.

How It Works

The architecture visible in the repo is a layered pipeline rather than a single giant model object: App entry -> ContentView / GameViewModel -> LifeSimulationOrchestrator -> domain systems -> GameState mutation -> EventEngine choice -> persistence save.

- App + UI: SwiftUI entry points create ContentView, which provides the planner shell, startup flow, event sheet, settings dialog, and bottom Age Up control.
- View model: GameViewModel owns presentation state, startup selection, tab choice, event visibility, and persistence interactions.
- State: GameState is the central aggregate model. It contains the player plus discrete domain states and history.
- Simulation service: LifeSimulationOrchestrator advances the year, applies actions, delegates domain logic, updates milestones, and requests the next event.
- Domain services: Trait, Policy, Education, Career, Finance, Housing, Relationship, Health, Asset, and Progress systems each encapsulate part of the yearly logic.
- Event content: EventEngine loads JSON events from the app bundle, filters them by age, cooldown, and requirements, then selects a weighted event.
- Persistence: PersistenceCoordinator stores a versioned save envelope, refreshes a backup, supports legacy fallbacks, and can reset all save files.

Compact data flow: the player commits yearly intent first, the orchestrator resolves systemic consequences second, and authored events personalize the year afterward.

How To Run

Verified app signal	The repo contains SwiftUI app entry points in OneLifeApp.swift and OneLifeBaseApp.swift.
Project/workspace file	Not found in repo.
README or setup guide	Not found in repo.
Exact build or run command	Not found in repo.

- Use an Apple development setup capable of launching a SwiftUI app.
- Run the app entry that hosts ContentView.
- Any concrete Xcode project, scheme, simulator target, or CLI build command is Not found in repo.

Current Architecture Manifesto

This manifesto is inferred from the code that exists today, not from external instructions.

- One year is the unit of play. The game batches life into annual turns and asks the player to commit to a yearly posture before outcomes are resolved.
- The interface is a planner. The UI is built around tabbed review, compressed status signals, repeatable section cards, and a single Age Up action.
- Life is modeled as connected domains. Education, career, finance, relationships, health, housing, assets, and progress are separate state buckets, but they are intentionally interdependent.
- Backstory is compressed so consequence can be the main gameplay. The origin phase summarizes childhood and lands the player at 14.
- Events are authored, systems are systemic. Event text and choice payloads come from JSON content, while long-term consequences are carried by code-defined systems and persistent state.
- Pressure is multi-domain, not cosmetic. Finance stress, school standing, work performance, relationship strain, health habits, and housing stability all show up as actionable parts of the same life framework.
- The game remembers selectively. Recent life is trimmed for display and history is budgeted for persistence, favoring readability and save-size discipline over exhaustive archival detail.
- Reliability is part of the framework. Backup saves, legacy save recovery, and DEBUG timing snapshots indicate that operational stability is already treated as a first-class concern.

Comparison To The Provided Instructions

The current codebase already shares a lot of DNA with the provided manifesto, but it is not a full implementation of that target state yet.

Instruction theme	Repo-based evaluation
Orchestrator and domain pattern	Strong alignment. The repo has a real LifeSimulationOrchestrator and distinct domain systems, and GameState stores domain-specific state instead of burying everything in a single mutable blob.
Rule of Two	Partial alignment. The architecture is domain-aware, but several system APIs still read from or mutate more than two domains in one pass. Examples include Finance, Health, and Asset updates taking multiple domain inputs.
Domain sovereignty	Mostly aligned. Logic is separated into named systems, but the yearly loop still coordinates a lot of cross-domain coupling centrally, so sovereignty is present but not absolute.
System hibernation	Not found in repo as an explicit registry or activation framework. The current yearly loop appears to run the systems sequentially every year, with only light contextual branching.
Policy modifier pattern	Aligned. A dedicated PolicySystem provides finance-region defaults plus education and healthcare modifiers rather than hard-coding those pressures everywhere.
UI continuity and glanceability	Strong alignment. The UI uses a consistent planner card system, fixed tab language, compressed top signals, color-coded tones, and a summary-first layout across domains.
No-scroll / one-handed emphasis	Partial alignment. The app clearly prioritizes compact overviews and a strong bottom action, but the startup flow already uses scroll views and some tab content can extend vertically.
Lazy NPC resolution	Not found in repo. Relationships are tracked in arrays of friends and romantic partners; no inactive NPC pool or deferred social simulation layer is visible.

Identity takeaway: the repo already reads as a domain-based annual life planner. The biggest gaps versus the provided instructions are stricter cross-domain boundaries, explicit system hibernation, and more formalized activation rules.

- Repo gaps worth keeping visible: public-facing app description, explicit audience statement, documented run instructions, and project/workspace or scheme metadata are all Not found in repo.

Bottom line: the current game framework is already coherent, and this document can serve as the present-tense identity baseline while the provided instructions define the next architectural tightening pass.